

Programme	M.Tech.	Semester	Win'25-26
Course	Computer Networks	Code	MACSE513
Faculty	Dr. R. Kumar	Slot	L11+L12

LAB: 8 - TCP & UDP

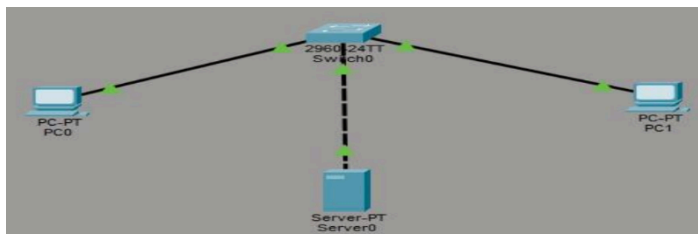
Submitted By: 25MCB1010, Patsamatla Bhaavya Ramita

1. TCP 3-Way Handshake (HTTP)

In this task, I used Cisco Packet Tracer to observe how the TCP 3-way handshake works for an HTTP connection, i.e., the SYN -> SYN-ACK -> ACK process between a client and a server.

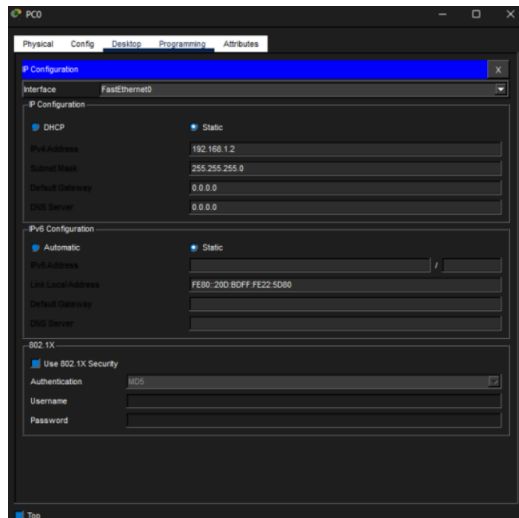
1.1 Network Setup

For this part, I created a simple network with 2 PCs, 1 switch, and 1 server, all in the same IP network and connected using copper straight-through cables.

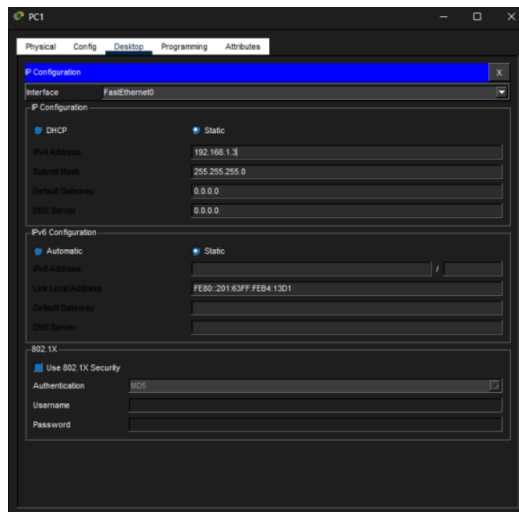


Network topology with 2 PCs, 1 switch and 1 server.

I manually configured the IP addresses for the devices so that the client and the server were in the same subnet, and the server had a static IP.

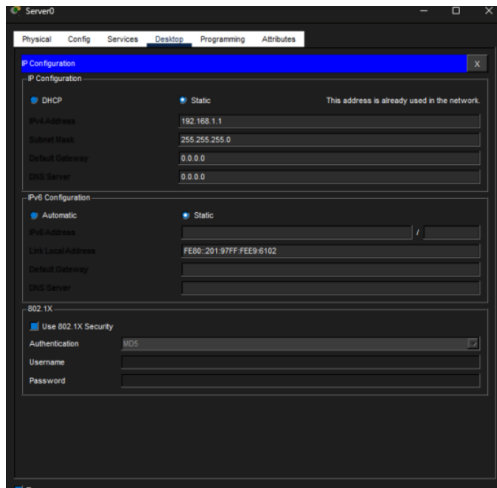


PC0 IP configuration.

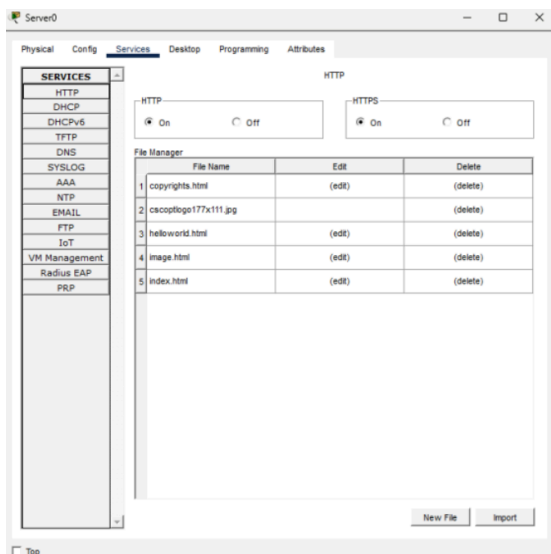


PC1 IP configuration.

I also configured the server with its IP and then enabled the HTTP service so that the client could access a web page from it.



Server IP configuration.



HTTP service enabled on the server.

Once basic IP and HTTP settings were done, I changed Packet Tracer from Realtime mode to Simulation mode and selected only TCP packets in the simulation filters so that I could clearly see only the TCP control packets

1.2 Running the Experiment

On the client PC (PC0), I opened the Web Browser application and typed the server's IP address in the address bar. This action caused the client to start the TCP connection setup towards the server so it could request the web page over HTTP.

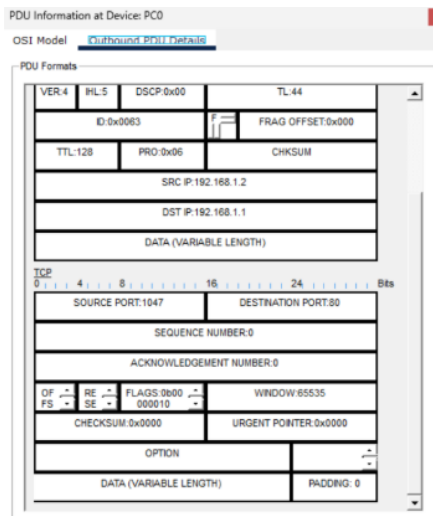
1.3 Observing the TCP 3-Way Handshake

- **Step 1 – SYN:**

The client first sends a TCP SYN packet to the server to request a new TCP session.

In this packet:

- Sequence Number = 0
- Acknowledgement Number = 0
- TCP Flag = SYN (0x002)



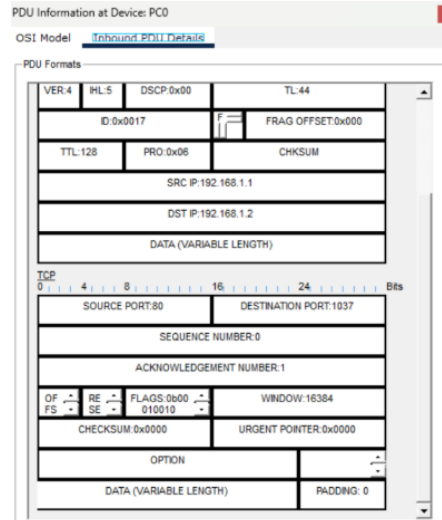
1st step of TCP 3-way handshake (SYN).

- **Step 2 – SYN-ACK:**

After receiving the SYN, the server replies with a SYN-ACK packet, which both acknowledges the client and indicates that the server is ready to establish the connection

- In this packet:

- Sequence Number = 0
- Acknowledgement Number = 1
- TCP Flags = SYN + ACK (0x012)



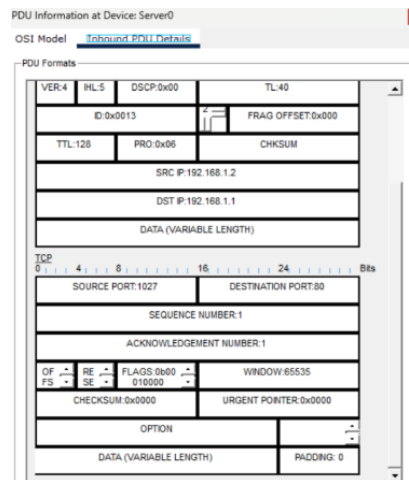
2nd step of TCP 3-way handshake (SYN-ACK).

- **Step 3 – ACK:**

The client then sends a final ACK back to the server, confirming the connection establishment.

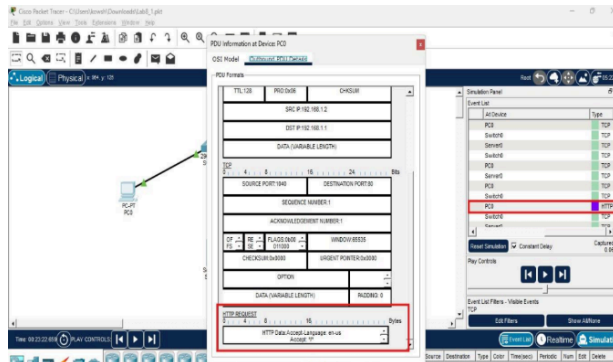
- In this packet:

- Sequence Number = 1
- Acknowledgement Number = 1
- TCP Flag = ACK (0x010)



3rd step of TCP 3-way handshake (ACK).

After these three steps, the TCP connection is considered established. Right after this handshake, the client sends an HTTP GET request, and the server responds with the web page, showing that a reliable connection is in place.



1.4 Result and Conclusion (Handshake)

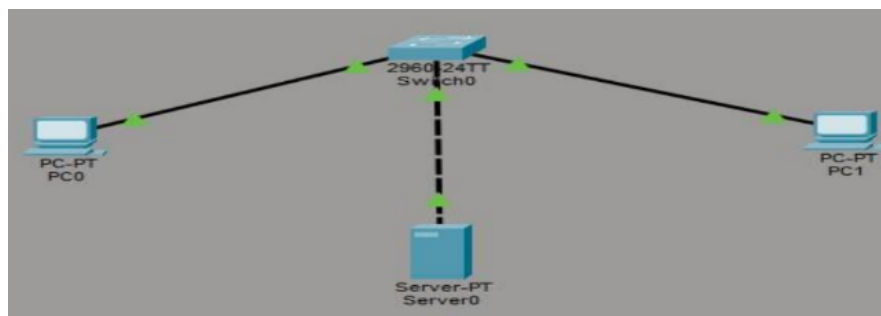
- I was able to clearly see the three steps: SYN -> SYN-ACK -> ACK between the client and the server.
- The handshake ensured that both sides were synchronized with sequence and acknowledgement numbers before any HTTP data was sent.

2. TCP 4-Way Connection Termination

In the second part, I observed how TCP closes a connection using a 4-way handshake (FIN and ACK packets) in Cisco Packet Tracer.

2.1 Network Setup

I used the same basic network as in the first experiment: two client PCs (PC0 and PC1), one server, and one switch, all in the same local network. The HTTP connection between client and server was already established before starting the termination observation.



Network topology showing 2 PCs, 1 switch and 1 server for termination.

Packet Tracer was again run in Simulation mode with only TCP packets enabled in the filters so that termination packets were easy to observe.

2.2 Running the Experiment

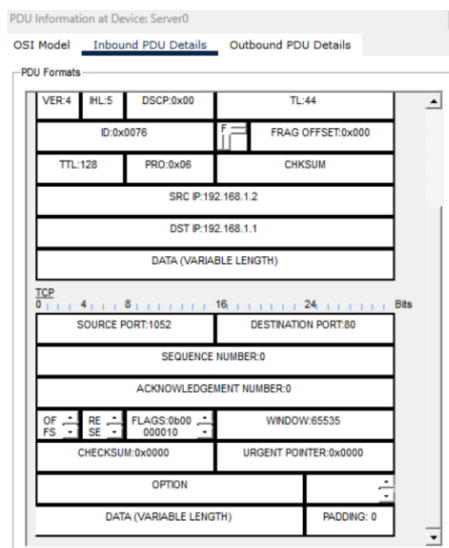
To trigger connection termination, I simply closed the web browser on the client side. Closing the browser caused the client to start the TCP connection teardown process.

2.3 Observing the TCP 4-Way Termination

- **Step 1 – FIN:**

One endpoint (for example, the client) sends a FIN packet to indicate that it has finished sending data and wants to close its side of the connection

- TCP Flag = FIN (0x001)

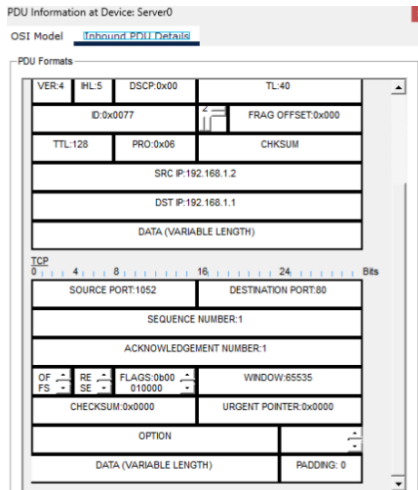


Step 1 – TCP FIN packet sent.

- **Step 2 – ACK for FIN:**

The other endpoint replies with an ACK packet to confirm it received the FIN request.

- TCP Flag = ACK (0x010)

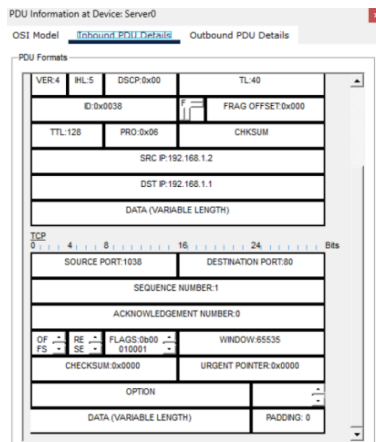


Step 2 – TCP ACK for FIN.

- **Step 3 – FIN-ACK from other side:**

After acknowledging the first FIN, the other side will also send its own FIN (often shown as FIN-ACK) when it is ready to close.

- TCP Flags = FIN + ACK (0x011)



Step 3 TCP FIN-ACK from the other endpoint.

- **Step 4 – Final ACK:**

Finally, the first endpoint sends an ACK back to confirm the second FIN, and then the connection is fully closed.

- TCP Flag = ACK (0x010)

Step 4 – Final TCP ACK and connection closed.

2.4 Result and Conclusion (Termination)

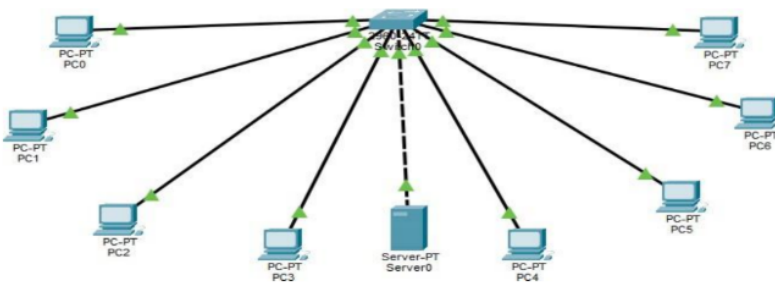
- I successfully observed how a TCP connection is closed using the 4-way handshake (FIN, ACK, FIN-ACK, final ACK).
- TCP ensures that both sides get a chance to finish sending data and acknowledge everything before the connection completely shuts down.

3. TCP Congestion Behaviour (Multiple PCs)

In this part, I studied how TCP behaves when multiple clients communicate with the same server at the same time and how congestion control comes into play.

3.1 Network Setup

I created a network with multiple client PCs (around 6-8), one server, and a switch, all connected by copper straight-through cables and kept in the same subnet. Each client PC got a unique IP address, while the server had a static IP and HTTP service enabled so it could handle multiple connections.

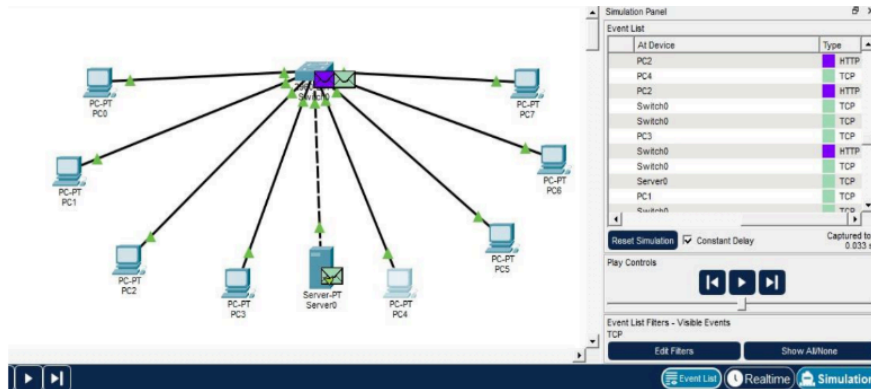


Network topology with multiple PCs, one server and one switch.

Simulation mode was enabled with only TCP packets selected in the filters to focus on TCP traffic during congestion.

3.2 Running the Experiment

I opened the web browser on all client PCs and made them access the server's IP address at roughly the same time. This created multiple simultaneous HTTP connections and generated heavy TCP traffic towards the server.



3.3 Observations

1. Sequence Numbers:

For each TCP connection, sequence numbers kept increasing as data was sent. Every connection maintained its own independent sequence space for reliable delivery

2. Acknowledgement Numbers:

ACK packets were sent by the receivers to confirm which bytes of data had been successfully received. The ACK numbers increased based on the sequence numbers of the data that arrived.

3. Retransmissions:

Under higher load, some packets were retransmitted, which usually indicates delay or packet loss due to congestion in the network.

4. Congestion Control:

TCP adjusted its sending rate depending on the network condition.

- The congestion window (cwnd) controlled how much unacknowledged data could be in the network.
- Mechanisms like Slow Start and Congestion Avoidance helped to increase the sending rate carefully and reduce it when congestion appeared.

5. Flow Control vs Congestion Control:

- Flow control made sure that the sender did not overwhelm the receiver, using the receive window size.
- Congestion control focused on not overloading the network itself by adjusting the transmission rate.

3.4 Result and Conclusion (Congestion)

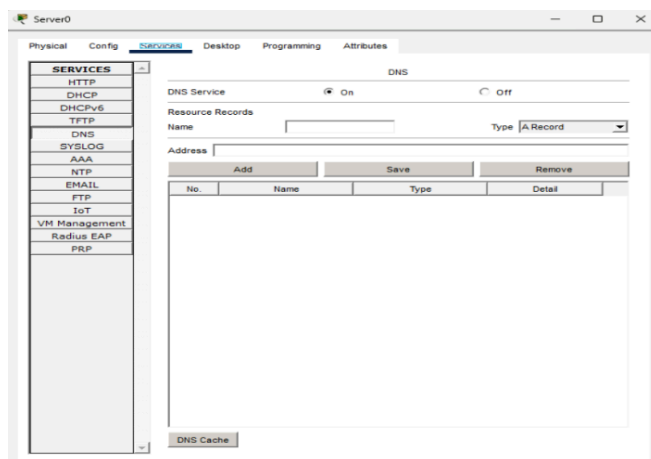
- TCP handled multiple simultaneous connections reliably, even when the network became busy.
- Retransmissions and window adjustments showed that congestion control and flow control were working to keep communication stable and efficient.

4. DNS Using UDP

In the final part, I observed how DNS works over UDP with a simple client-server setup in Packet Tracer.

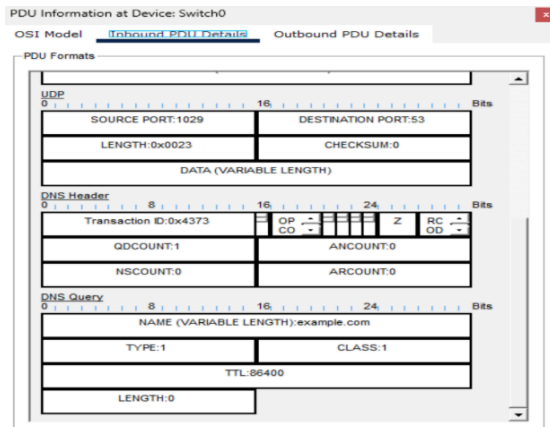
4.1 Network Setup

The network had one client PC, one server and one switch, all connected using copper straight-through cables and configured in the same IP network. IP addresses were manually assigned, with the server getting a static IP, to make sure DNS communication worked properly.



Basic DNS network setup (PC, server and switch).

On the server side, I enabled the DNS service and created a DNS record that mapped a domain name to the server's IP address so that the server could respond to DNS queries.



DNS service configuration on the server.

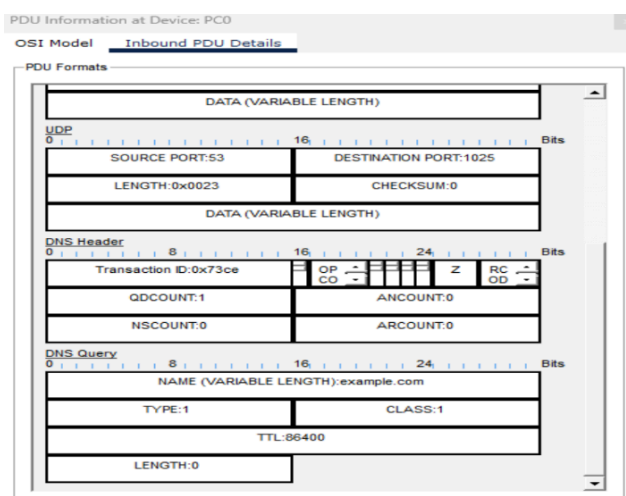
4.2 Running the Experiment

On the client PC, I opened the Command Prompt and used the `nslookup` command to query the DNS server for the configured domain name. This generated DNS request and response packets between the client and the DNS server.

4.3 Observing DNS over UDP

- **DNS Request:**

The client sent a DNS request packet to the DNS server using UDP, with destination port 53 (standard DNS port). The packet carried the domain name that needed to be resolved.



DNS request packet sent by client using UDP.

- **DNS Response:**

The DNS server replied with a DNS response packet, containing the IP address that matched the requested domain name. This response was also sent using UDP, without any connection-oriented handshake.

4.4 Result and Conclusion (DNS)

- I was able to see a complete DNS query and response cycle using UDP.
- The domain name was successfully resolved to an IP address using a simple request-response mechanism, without establishing a TCP-style connection.